

94. Sticky flag (y): kisi position par search

(y) flag string me diya gaya (exact) position par search karne deta hai.

Kaam kahan aata hai?

Regexps ka ek common kaam "lexical analysis": text (jaise programming language ka code) padhkar uske structural elements dhundhna. Ismein ek aam kaam: **kisi position par kuch padhna**.

Maano string `let varName = "value"` hai aur position `4` se variable name padhna hai.

- `str.match(/\w+/)` sirf pehla word (`let`) dega. Nahi.
- `(g)` flag se `str.match(/\w+/g)` saare words dega. Nahi.

Position 4 par exactly search kaise karein?

`regexp.exec` aur `lastIndex`

`(g)` aur `(y)` ke bina `regexp.exec(str)` `str.match(regexp)` jaisa hi hai (pehla match). Par `(g)` **flag ho** to search `regexp.lastIndex` property ki position se **shuru** hoti hai, aur match milne par `lastIndex` match ke **turant baad** ki position par set ho jaata hai.

Yaani `lastIndex` search ka starting point hai, jo har `exec` call ke baad "pichhle match ke baad" ho jaata hai. Isse baar baar `exec` call karke ek ek karke saare matches milte hain:

```
js

let str = 'let varName';
let regexp = /\w+/g;

regexp.lastIndex;           // 0

let word1 = regexp.exec(str);
word1[0];                   // let
regexp.lastIndex;           // 3

let word2 = regexp.exec(str);
word2[0];                   // varName
regexp.lastIndex;           // 11

let word3 = regexp.exec(str); // null
regexp.lastIndex;           // 0 (search khatam par reset)
```

Loop me:

js

```
let result;
while (result = regexp.exec(str)) {
  alert(`Found ${result[0]} at position ${result.index}`);
}
```

Ye `str.matchAll` ka alternative hai, thoda zyada control ke saath.

Ab apne task par: `lastIndex` khud 4 set kar do:

js

```
let regexp = /\w+/g; // "g" ke bina lastIndex ignore hota hai
regexp.lastIndex = 4;
regexp.exec('let varName = "value"'); // varName
```

Par ruko: `exec` `lastIndex` se **shuru** karke aage tak dhundhta hai. Agar `lastIndex` par word na ho par uske baad kahin ho to wo mil jaata hai:

js

```
regexp.lastIndex = 3; // position 3 par space hai
let word = regexp.exec(str);
word[0]; // varName
word.index; // 4 (aage mila)
```

Lexical analysis jaise kaamon me ye galat hai. Hume match **exactly** given position par chahiye. Wahi `y` flag karta hai.

`y` flag

`y` flag `regexp.exec` ko exactly `lastIndex` par search karne deta hai, "wahan se shuru karke" nahi.

js

```
let regexp = /\w+/y;

regexp.lastIndex = 3;
regexp.exec(str); // null (position 3 par space hai, word nahi)

regexp.lastIndex = 4;
regexp.exec(str); // varName
```

Ye sirf sahi hi nahi, **performance** me bhi fayda deta hai. Bade text me koi match na ho to (g) wala search poore end tak jaakar kuch nahi milta (bahut time), jabki (y) sirf exact position check karta hai. Lexical analysis me aksar bahut saari searches exact position par hoti hain ki "yaha kya hai?", to (y) sahi implementation aur achhi performance ki chaabi hai.